

HW_02

Henry Traynor

Due: 5:00pm, 31 August

Assignment

Create a self-contained R script or Rmarkdown file to do the following:

1. Import the Canada Warbler data (`cawa_data_2017_occu.csv`) and compute basic summary stats like we did earlier.
 - Data are “point count data” from North Carolina.
 - Each 10-min survey was divided into 4, 2.5-min intervals

```
# data import and column rename
setwd("C:/Users/henry/Desktop/WILD8390/Homework/HW02")
warbler_df <- read.csv("cawa_data_2017_occu.csv")
colnames(warbler_df) <- c("Site", "obs1", "obs2", "obs3", "obs4")

# background values
nSites <- length(warbler_df[,1])
nVisits <- 4

# number of detections for each site
siteDets <- rowSums(warbler_df[,2:5])
table(siteDets)
```

```
## siteDets
##  0  2  3  4
## 97  1  7  4
```

```
# naive occupancy (proportion of sites with >0 observations)
naiveOccupancy <- sum(siteDets>0)/nSites
naiveOccupancy
```

```
## [1] 0.1100917
```

2. Fit the single-season (static) occupancy model using `unmarked` and `JAGS`.

```
library(unmarked)

# convert to 'unmarkedFrame' data structure
# remove site names for summary to be cleaner
umf <- unmarkedFrameOccu(y=warbler_df[,2:5])
summary(umf)
```

```
## unmarkedFrame Object
##
## 109 sites
## Maximum number of observations per site: 4
## Mean number of observations per site: 4
## Sites with at least one detection: 12
##
## Tabulation of y observations:
##    0    1
## 397  39
```

```
# model fitting
fm <- occu(~1 ~1, umf)
summary(fm)
```

```
##
## Call:
## occu(formula = ~1 ~ 1, data = umf)
##
## Occupancy (logit-scale):
##   Estimate    SE      z  P(>|z|)
##    -2.09 0.306 -6.82 8.89e-12
##
## Detection (logit-scale):
##   Estimate    SE      z  P(>|z|)
##     1.46 0.373 3.91 9.06e-05
##
## AIC: 125.8796
## Number of sites: 109
```

```
backTransform(fm, type="state")
```

```
## Backtransformed linear combination(s) of Occupancy estimate(s)
##
##   Estimate    SE LinComb (Intercept)
##    0.11 0.03   -2.09             1
##
## Transformation: logistic
```

```
backTransform(fm, type="det")
```

```
## Backtransformed linear combination(s) of Detection estimate(s)
##
##   Estimate    SE LinComb (Intercept)
##    0.811 0.057    1.46             1
##
## Transformation: logistic
```

```
# posterior probs Pr(z_1 | y_i)
z.post <- ranef(fm)
# extract posterior means
```

```
psi_realized <- bup(z.post, stat="mean")
```

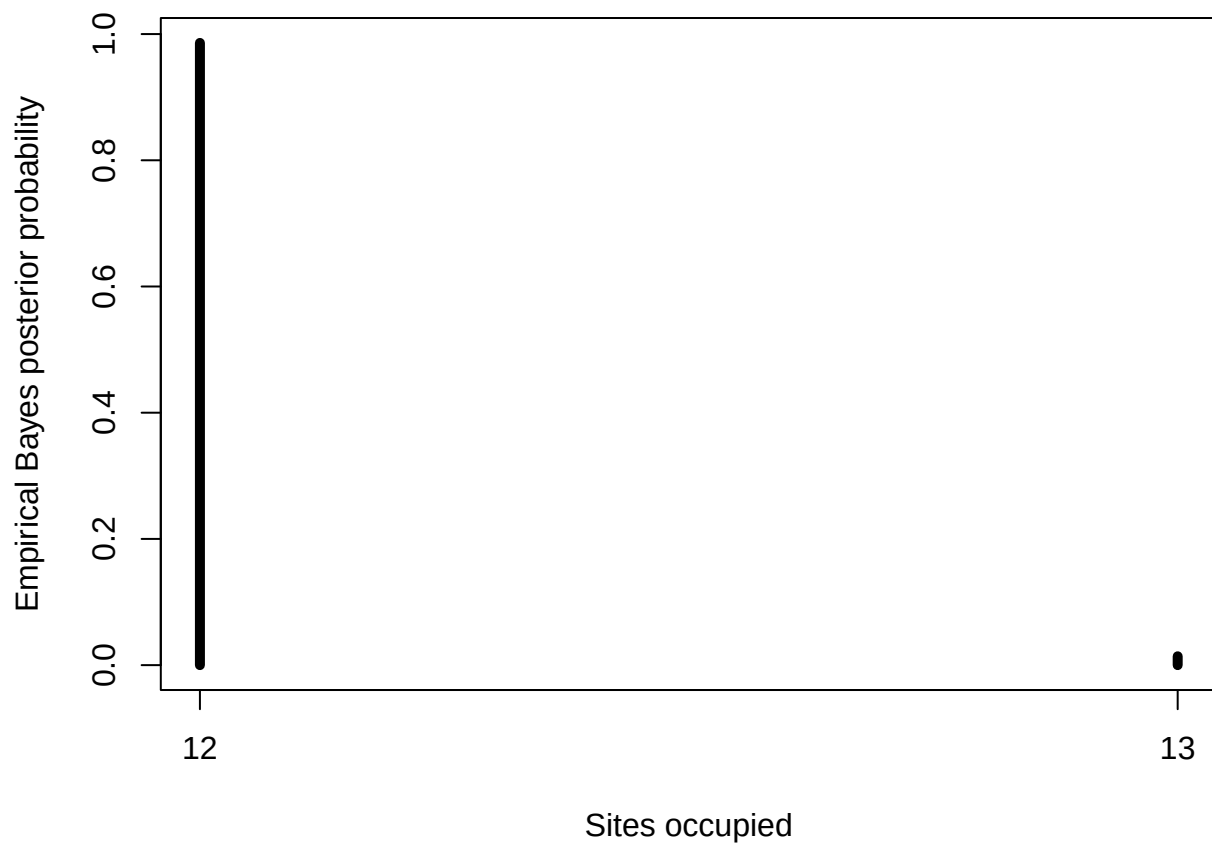
```
round(data.frame(y=warbler_df[,2:5], psi_expected=predict(fm, type="state")[,1], psi_realized), 3)
```

##	y.obs1	y.obs2	y.obs3	y.obs4	psi_expected	psi_realized
## 1	1	1	1	1	0.11	1
## 2	0	0	0	0	0.11	0
## 3	0	0	0	0	0.11	0
## 4	0	0	0	0	0.11	0
## 5	0	0	0	0	0.11	0
## 6	0	0	0	0	0.11	0
## 7	1	0	1	1	0.11	1
## 8	0	0	0	0	0.11	0
## 9	1	1	1	1	0.11	1
## 10	0	0	0	0	0.11	0
## 11	0	0	0	0	0.11	0
## 12	0	0	0	0	0.11	0
## 13	0	0	0	0	0.11	0
## 14	0	0	0	0	0.11	0
## 15	0	0	0	0	0.11	0
## 16	0	0	0	0	0.11	0
## 17	0	0	0	0	0.11	0
## 18	0	0	0	0	0.11	0
## 19	0	0	0	0	0.11	0
## 20	0	0	0	0	0.11	0
## 21	0	0	0	0	0.11	0
## 22	0	0	0	0	0.11	0
## 23	0	0	0	0	0.11	0
## 24	0	0	0	0	0.11	0
## 25	0	0	0	0	0.11	0
## 26	0	0	0	0	0.11	0
## 27	1	1	1	1	0.11	1
## 28	0	0	0	0	0.11	0
## 29	0	0	0	0	0.11	0
## 30	0	0	0	0	0.11	0
## 31	0	0	0	0	0.11	0
## 32	0	0	0	0	0.11	0
## 33	0	0	0	0	0.11	0
## 34	0	0	0	0	0.11	0
## 35	0	0	0	0	0.11	0
## 36	0	0	0	0	0.11	0
## 37	1	1	0	0	0.11	1
## 38	0	0	0	0	0.11	0
## 39	0	0	0	0	0.11	0
## 40	0	0	0	0	0.11	0
## 41	0	0	0	0	0.11	0
## 42	0	0	0	0	0.11	0
## 43	0	0	0	0	0.11	0
## 44	0	0	0	0	0.11	0
## 45	0	0	0	0	0.11	0
## 46	0	0	0	0	0.11	0
## 47	0	0	0	0	0.11	0
## 48	0	0	0	0	0.11	0

## 49	0	0	0	0	0.11	0
## 50	0	0	0	0	0.11	0
## 51	0	0	0	0	0.11	0
## 52	0	0	0	0	0.11	0
## 53	0	0	0	0	0.11	0
## 54	0	0	0	0	0.11	0
## 55	0	0	0	0	0.11	0
## 56	0	0	0	0	0.11	0
## 57	0	0	0	0	0.11	0
## 58	0	0	0	0	0.11	0
## 59	0	0	0	0	0.11	0
## 60	0	0	0	0	0.11	0
## 61	0	0	0	0	0.11	0
## 62	0	0	0	0	0.11	0
## 63	0	0	0	0	0.11	0
## 64	0	0	0	0	0.11	0
## 65	0	0	0	0	0.11	0
## 66	0	0	0	0	0.11	0
## 67	0	0	0	0	0.11	0
## 68	0	0	0	0	0.11	0
## 69	0	0	0	0	0.11	0
## 70	0	0	0	0	0.11	0
## 71	0	0	0	0	0.11	0
## 72	0	0	0	0	0.11	0
## 73	0	0	0	0	0.11	0
## 74	0	0	0	0	0.11	0
## 75	0	0	0	0	0.11	0
## 76	0	0	0	0	0.11	0
## 77	0	0	0	0	0.11	0
## 78	0	1	1	1	0.11	1
## 79	0	0	0	0	0.11	0
## 80	0	0	0	0	0.11	0
## 81	0	0	0	0	0.11	0
## 82	0	1	1	1	0.11	1
## 83	0	0	0	0	0.11	0
## 84	0	0	0	0	0.11	0
## 85	0	1	1	1	0.11	1
## 86	0	0	0	0	0.11	0
## 87	0	0	0	0	0.11	0
## 88	1	1	1	0	0.11	1
## 89	0	1	1	1	0.11	1
## 90	0	0	0	0	0.11	0
## 91	0	0	0	0	0.11	0
## 92	0	0	0	0	0.11	0
## 93	0	0	0	0	0.11	0
## 94	0	0	0	0	0.11	0
## 95	0	0	0	0	0.11	0
## 96	0	0	0	0	0.11	0
## 97	0	0	0	0	0.11	0
## 98	0	0	0	0	0.11	0
## 99	0	0	0	0	0.11	0
## 100	0	0	0	0	0.11	0
## 101	0	0	0	0	0.11	0
## 102	0	0	0	0	0.11	0

```
## 103      0      0      0      0      0.11      0
## 104      1      0      1      1      0.11      1
## 105      0      0      0      0      0.11      0
## 106      0      0      0      0      0.11      0
## 107      1      1      1      1      0.11      1
## 108      0      0      0      0      0.11      0
## 109      0      0      0      0      0.11      0
```

```
nsim <- 1000
sites.occupied.post <- predict(z.post, func=sum, nsim=nsim)
par(mai=c(0.9,0.9,0.1,0.1))
plot(table(sites.occupied.post)/nsim, lwd=5, xlab="Sites occupied",
      ylab="Empirical Bayes posterior probability")
```



```
psi.realized.estimate <- median(sites.occupied.post)
psi.realized.estimate
```

```
## [1] 12
```

```
quantile(sites.occupied.post, prob=c(0.025, 0.975))
```

```
## 2.5% 97.5%
## 12 12
```

```
library(jagsUI)
library(rjags)
```

```
## Loading required package: coda
```

```
##
```

```
## Attaching package: 'coda'
```

```
## The following object is masked from 'package:jagsUI':
```

```
##
```

```
##      traceplot
```

```
## Linked to JAGS 4.3.1
```

```
## Loaded modules: basemod,bugs
```

```
string <- "
model {
  psi ~ dunif(0,1)
  p ~ dunif(0,1)

  for(i in 1:nSites) {
    z[i] ~ dbern(psi)
    for(j in 1:nVisits) {
      y[i,j] ~ dbern(z[i]*p)
    }
  }

  sitesOccupied <- sum(z)
}
"

jags.data <- list(y=warbler_df[,2:5],
                 nSites = nSites,
                 nOccasions=nVisits)
jags.inits <- function() {
  list(psi=runif(1), p = runif(1), z=rep(1,nSites))
}
jags.pars <- c("psi", "p", "sitesOccupied")

jags.post.samples <- jags.basic(data=jags.data, inits=jags.inits,
                               parameters.to.save=jags.pars,
                               model.file="occupancy-model.jag",
                               n.chains=3, n.adapt=100, n.burnin=0,
                               n.iter=2000, parallel=TRUE)
```

```
##
```

```
## Processing function input.....
```

```
##
```

```
## Converted data.frame y to matrix
```

```
##
```

```
## Done.
##
## Beginning parallel processing using 3 cores. Console output will be suppressed.
##
## Parallel processing completed.
##
## MCMC took 0.016 minutes.
```

```
summary(jags.post.samples)
```

```
##
## Iterations = 101:2100
## Thinning interval = 1
## Number of chains = 3
## Sample size per chain = 2000
##
## 1. Empirical mean and standard deviation for each variable,
##    plus standard error of the mean:
##
##              Mean      SD Naive SE Time-series SE
## deviance      47.6982 2.65784 0.0343126      0.0485718
## p              0.7976 0.05709 0.0007370      0.0010052
## psi           0.1174 0.03028 0.0003909      0.0003928
## sitesOccupied 12.0335 0.18987 0.0024512      0.0027647
##
## 2. Quantiles for each variable:
##
##              2.5%      25%      50%      75%      97.5%
## deviance      46.32836 46.4253 46.7828 47.7433 58.5463
## p              0.67093 0.7615 0.8025 0.8374 0.8967
## psi           0.06423 0.0957 0.1156 0.1361 0.1838
## sitesOccupied 12.00000 12.0000 12.0000 12.0000 13.0000
```

```
# run this before if having issues:
# while (!is.null(dev.list())) dev.off()
# run all 3 lines selected together
png("jags_plot.png", width = 500, height = 800)
plot(jags.post.samples)
dev.off()
```

```
## pdf
## 2
```

3. Create a data.frame to compare the estimates of ψ , p , and `nSitesOccupied` from the two analyses. Compare point estimates and 95% CIs. Describe any differences you see in the estimates.

```
# I had to do some messing around here to get things to pull dynamically
# this is the way that made the most sense to me since jags.post.samples is an unnamed list
library(coda)

# parms for df rows
```

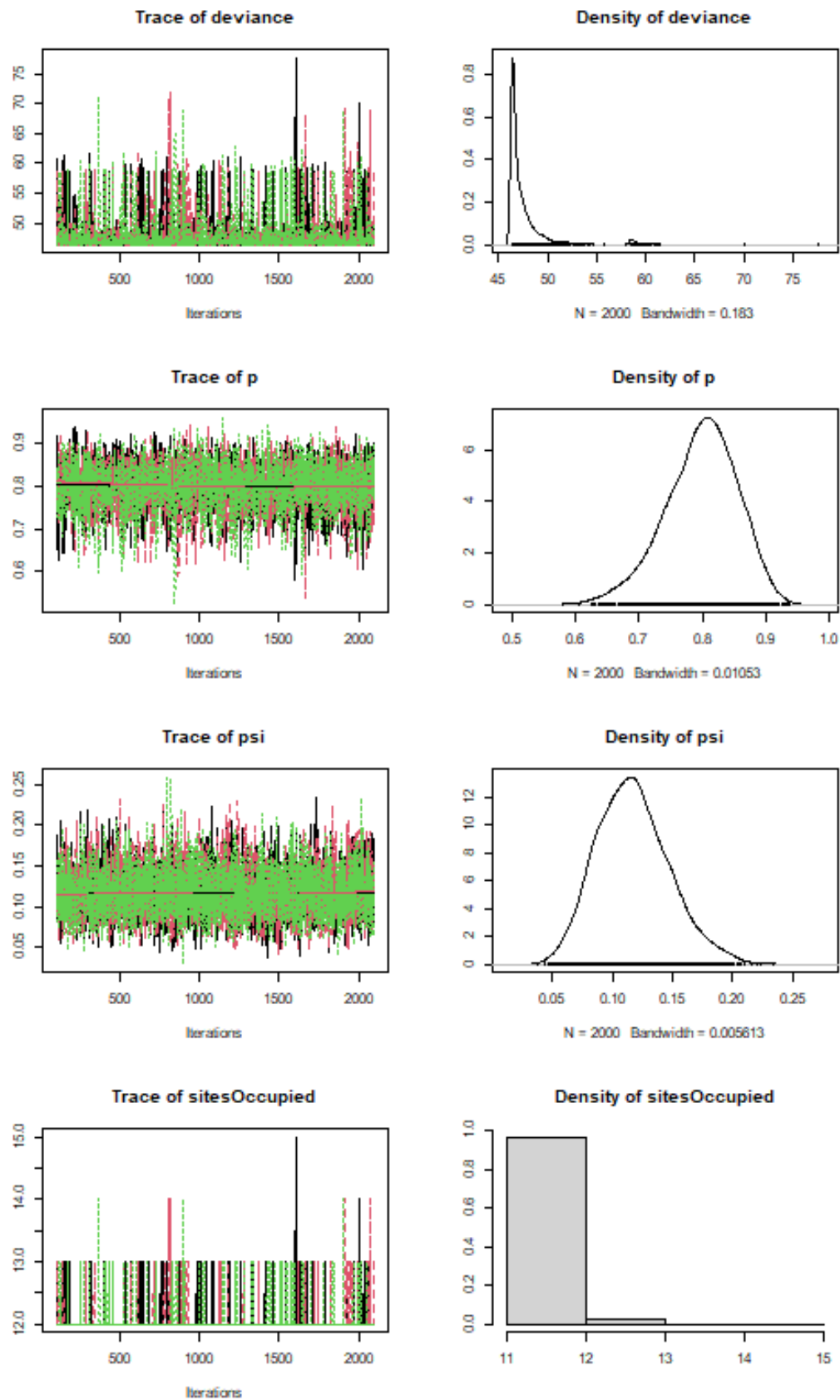


Figure 1: Traceback plot

```

parameters <- c("psi", "p", "sitesOccupied")

# backtransform estimates
unmarked_psi <- backTransform(fm, type = "state")@estimate
unmarked_p <- backTransform(fm, type = "det")@estimate

# pulls CIs from fitted model
psi_CI <- confint(fm, type = "state")
p_CI <- confint(fm, type = "det")

#back to probability scale from logit scale
unmarked_psi_CI <- plogis(psi_CI)
unmarked_p_CI <- plogis(p_CI)

unmarked_sites <- median(sites.occupied.post)
unmarked_sites_CI <- quantile(sites.occupied.post, c(0.025, 0.975))

# JAGS output was giving me an unnamed list so i had to pull this out weird
# now we can combine all samples in one matrix for everything
jags_mcmc <- as.mcmc.list(jags.post.samples)
jags_samples <- as.matrix(jags_mcmc)

jags_estimates <- sapply(parameters, function(x) {
  mean(jags_samples[, x])
})

jags_CIs <- t(sapply(parameters, function(x) {
  quantile(jags_samples[, x], c(0.025, 0.975))
})))

# now through all that in a dataframe
df_compare <- data.frame(
  Parameter = parameters,
  unmarked_Estimate = c(
    unmarked_psi,
    unmarked_p,
    unmarked_sites
  ),
  unmarked_Lower = c(
    unmarked_psi_CI[1],
    unmarked_p_CI[1],
    unmarked_sites_CI[1]
  ),
  unmarked_Upper = c(
    unmarked_psi_CI[2],
    unmarked_p_CI[2],
    unmarked_sites_CI[2]
  ),
  JAGS_Estimate = jags_estimates,
  JAGS_Lower = jags_CIs[, 1],
  JAGS_Upper = jags_CIs[, 2]
)

```

```

# round for presentation
library(dplyr)

##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##   filter, lag

## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union

table_compare <- df_compare %>%
  mutate(
    unmarked = sprintf(
      "%.3f (%.3f-%.3f)",
      unmarked_Estimate,
      unmarked_Lower,
      unmarked_Upper
    ),
    JAGS = sprintf(
      "%.3f (%.3f-%.3f)",
      JAGS_Estimate,
      JAGS_Lower,
      JAGS_Upper
    )
  ) %>%
  select(Parameter, unmarked, JAGS)

knitr::kable(
  table_compare,
  caption = "Comparison of models",
  col.names = c("Parameter", "unmarked", "JAGS"),
  align = c("l", "c", "c")
)

```

Table 1: Comparison of models

	Parameter	unmarked	JAGS
psi	psi	0.110 (0.064–0.184)	0.117 (0.064–0.184)
p	p	0.811 (0.675–0.899)	0.798 (0.671–0.897)
sitesOccupied	sitesOccupied	12.000 (12.000–12.000)	12.034 (12.000–13.000)

The estimates from `unmarked` and JAGS analysis were nearly identical. Occupancy (ψ) estimates were within one hundredth of each other, and CIs overlapped. Detection probabilities were also quite similar. The differences in CIs for the number of sites occupied come from maximum likelihood versus Bayesian estimation. The `unmarked` analysis had a median estimate of 12 occupied sites with a 95% interval of [12, 12], while the JAGS analysis had a posterior mean of 12.04 sites with a 95% credible interval of 12–13. Thus, both methods support 12 occupied sites, although JAGS assigns a small probability to 13 occupied sites.